

1 INTRODUCCIÓN

La integración entre aplicaciones requiere mecanismos estandarizados que permitan garantizar la seguridad, trazabilidad y control de acceso durante el intercambio de información.

El SISSEG V3.0 se compone de diversos componentes, tanto para conexiones Legacy como para integraciones actuales. Cuenta con un módulo de autenticación Keycloak, un módulo de autorización SISSEG-backend y un componente de acceso a los sistemas denominado SISSEG-frontend. En conjunto, forman el SISSEG V3.0.

El presente documento establece los lineamientos técnicos para el consumo de servicios entre aplicaciones, definiendo un modelo común de autenticación, autorización y registro de trazabilidad basado en tokens emitidos por la plataforma corporativa de identidad y en servicios centralizados de validación.

Estos lineamientos buscan asegurar que únicamente aplicaciones autorizadas puedan acceder a los recursos expuestos por otras aplicaciones, manteniendo un control uniforme sobre los permisos y la auditoría de los consumos realizados.

2 OBJETIVO

Establecer el mecanismo estándar para la comunicación entre aplicaciones, mediante el uso de tokens de autenticación, validación centralizada de credenciales, control de acceso basado en roles y registro de trazabilidad de las operaciones realizadas.

3 ALCANCE

El presente documento aplica a todas las aplicaciones institucionales que expongan o consuman servicios de otras aplicaciones dentro de la organización.

Comprende:

- Consumo de servicios entre aplicaciones.
- Autenticación mediante tokens.
- Validación centralizada de identidad.
- Validación de roles de acceso.
- Registro de trazabilidad de solicitudes.

- Manejo de respuestas de autorización.

No comprende:

- Integraciones con proveedores externos no institucionales.
 - Mecanismos de autenticación interactivos orientados a usuarios finales.
-

4 COMPONENTES INVOLUCRADOS

4.1 APLICACIÓN CONSUMIDORA (APLICACIÓN A)

Aplicación que requiere consumir un servicio expuesto por otra aplicación.

Responsabilidades:

- Obtener un token válido.
- Enviar el token en cada solicitud.
- Gestionar las respuestas del servicio consumido.

4.2 APLICACIÓN PROVEEDORA (APLICACIÓN B)

Aplicación que expone uno o más servicios para consumo por otras aplicaciones.

Responsabilidades:

- Recibir solicitudes.
- Validar el token recibido.
- Validar los permisos asociados.
- Registrar la trazabilidad del consumo.
- Autorizar o rechazar la solicitud.

4.3 PROVEEDOR DE IDENTIDAD

Componente encargado de emitir los tokens de acceso utilizados por las aplicaciones.

Responsabilidades:

- Gestionar clientes.
- Gestionar roles.
- Emitir tokens de autenticación.
- Actualizar los tokens de autenticación.

4.4 SERVICIO PTSD_TOKEN_VALIDAR

Servicio corporativo encargado de validar la información de seguridad contenida en los tokens recibidos.

Responsabilidades:

- Validar autenticidad.
- Validar vigencia.
- Retornar el resultado de la validación.

4.5 SERVICIO DE TRAZABILIDAD

Servicio encargado de almacenar el historial de consumos realizados entre aplicaciones.

Responsabilidades:

- Registrar solicitudes con la información de auditoría.

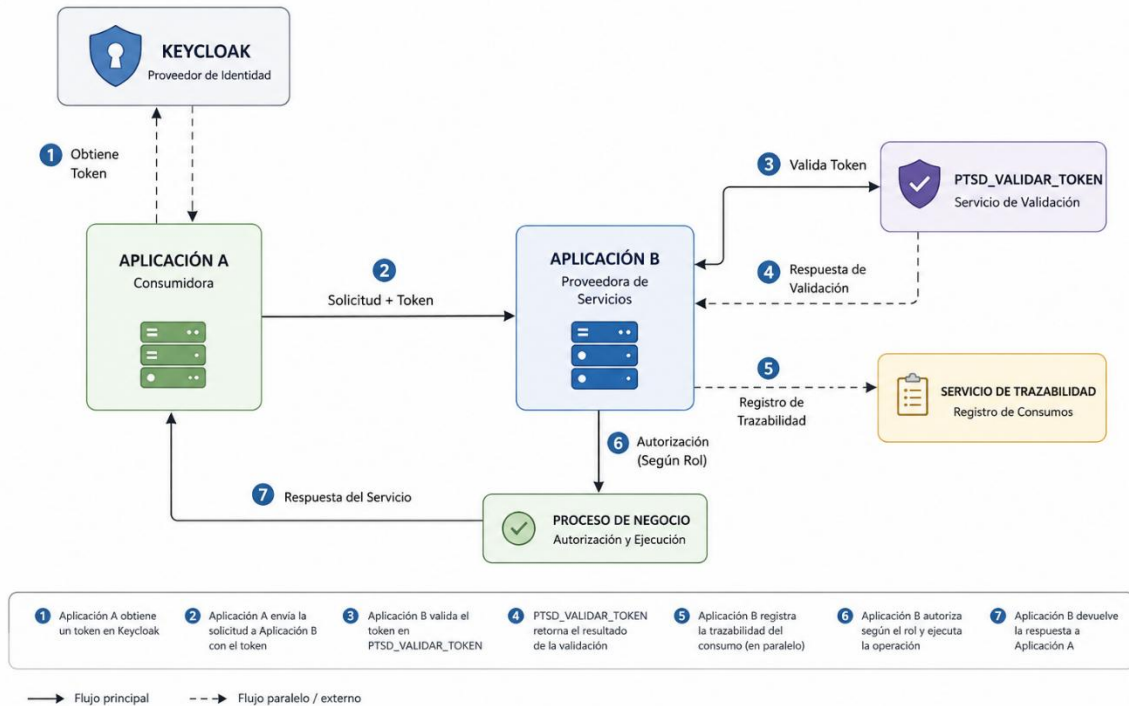
5 ARQUITECTURA DE REFERENCIA

La comunicación entre aplicaciones deberá seguir una arquitectura basada en validación centralizada.

La Aplicación A enviará un token de acceso junto con la solicitud de consumo.

La Aplicación B validará dicho token mediante el servicio PTSD_TOKEN_VALIDAR y verificará que el rol contenido en el token tenga autorización para ejecutar la operación solicitada.

De forma paralela al proceso de autorización, la Aplicación B deberá registrar la información del consumo en el servicio corporativo de trazabilidad.

ARQUITECTURA DE CONSUMO ENTRE APLICACIONES

6 FLUJO ESTÁNDAR DE CONSUMO

6.1 PASO 1: OBTENCIÓN DEL TOKEN

La Aplicación A obtiene un token emitido por el proveedor de identidad institucional.

El token deberá contener la información necesaria para identificar:

- Cliente.
- Realm.
- Roles asociados.
- Fecha de expiración.

6.2 PASO 2: INVOCACIÓN DEL SERVICIO

La Aplicación A invoca un servicio de la Aplicación B enviando el token en la cabecera de autorización.

6.3 PASO 3: VALIDACIÓN DEL TOKEN

La Aplicación B recibe la solicitud y consume el servicio PTSD_TOKEN_VALIDAR.

La validación deberá verificar:

- Integridad del token.
- Vigencia.
- Cliente vigente.
- Realm permitido.

6.4 PASO 4: EVALUACIÓN DE ROLES

Una vez recibida la respuesta de PTSD_TOKEN_VALIDAR, la Aplicación B evaluará si alguno de los roles retornados cumple con los permisos requeridos para la operación solicitada.

6.5 PASO 5: REGISTRO DE TRAZABILIDAD

De forma paralela a la evaluación de autorización, la Aplicación B deberá enviar información al servicio de trazabilidad.

Como mínimo deberán registrarse:

- Fecha y hora.
- Aplicación consumidora.
- Servicio invocado.
- Cliente identificado.
- Rol evaluado.
- Resultado de validación.
- Estado final de la operación.

6.6 PASO 6: AUTORIZACIÓN

Si el rol cumple con las reglas definidas para el servicio solicitado:

- Se procesa la operación.
- Se devuelve una respuesta exitosa.

Si el rol no cumple las reglas establecidas:

- Se rechaza la solicitud.
 - Se devuelve la respuesta de acceso denegado.
-

7 REGLAS DE AUTORIZACIÓN

Toda autorización deberá basarse en los roles contenidos en el token.

Las aplicaciones proveedoras deberán de validar el rol contenido en el token.

Toda operación deberá validar:

- Token válido.
- Cliente vigente.
- Rol autorizado.
- Estado correcto de la validación.

El acceso deberá ser denegado cuando cualquiera de estas validaciones falle.

8 REGLAS DE TRAZABILIDAD

Toda solicitud recibida deberá generar un registro de trazabilidad.

La trazabilidad deberá permitir identificar:

- Quién consumió el servicio.
- Qué servicio fue consumido.
- Cuándo ocurrió el consumo.
- Qué rol fue evaluado.
- Cuál fue el resultado de la autorización.

La información registrada deberá conservarse según las políticas institucionales vigentes.

9 MANEJO DE ERRORES

Las aplicaciones deberán responder adecuadamente ante los siguientes escenarios:

Rol insuficiente	Acceso denegado
Error en PTSD_TOKEN_VALIDAR	Error de validación
Error en trazabilidad	Registrar incidente y continuar según política definida

10 RESPONSABILIDADES

10.1 APLICACIÓN CONSUMIDORA

- Obtener credenciales válidas.
- Enviar el token correspondiente, al consumer un servicio.
- Gestionar adecuadamente las respuestas recibidas.

10.2 APLICACIÓN PROVEEDORA

- Ejecutar las validaciones de seguridad.
- Evaluar roles autorizados.
- Registrar la trazabilidad.
- Aplicar las políticas de acceso.

10.3 ADMINISTRADOR DE IDENTIDAD

- Gestionar usuarios de aplicación.
- Gestionar roles.
- Mantener la configuración de autenticación.

11 CONSIDERACIONES DE SEGURIDAD

- Todo consumo deberá realizarse mediante HTTPS.
 - Los tokens no deberán almacenarse en registros de aplicación.
 - Los roles deberán otorgarse bajo el principio de mínimo privilegio.
 - La validación deberá realizarse antes de ejecutar cualquier lógica de negocio.
 - Los errores no deberán exponer información sensible.
-

12 ANEXOS

12.1 ANEXO A: INFORMACIÓN DE INTEGRACIÓN

Concepto	Valor
----------	-------

Realm	OSIINTEGRACION
-------	----------------

Proveedor de Identidad	SISSEG - Keycloak
------------------------	-------------------

Protocolo	OAuth 2.0 / OpenID Connect
------------------	----------------------------

Tipo de Token	Bearer Token (JWT)
----------------------	--------------------

Mecanismo de Autenticación	Password Grant o Client Credentials (según corresponda)
-----------------------------------	---

12.2 ANEXO B: URLS DE REFERENCIA

Descripción	URL
--------------------	------------

Realm de autenticación	https://srvdesakclk.osineergmin.gob.pe/realms/OSIINTEGRACION
-------------------------------	---

Endpoint de obtención de token	https://srvdesakclk.osineergmin.gob.pe/realms/OSIINTEGRACION/protocol/openid-connect/token
---------------------------------------	---

Servicio de validación de token	https://srvdesaptsd.osineergmin.gob.pe/ptsd/api/tokenValidar/token/validate
--	---

Servicio de trazabilidad	[URL_SERVICIO_TRAZABILIDAD]
---------------------------------	-----------------------------

12.3 ANEXO C: CONSUMIR SERVICIO DE VALIDACIÓN DE TOKEN

URL: <https://srvdesaptsd.osineergmin.gob.pe/ptsd/api/tokenValidar/token/validate>

Tipo de método: GET

Parámetro: (Authorization) token

12.4 ANEXO D: EJEMPLO DE APLICACIÓN CONSUMIDORA

La aplicación consumidora genera un token al momento de querer consumir la aplicación que provee servicios.

```
public String obtenerAccessToken(String username, String password) {  
    try {  
        String body = restClient.post()  
            .uri(properties.keycloak().tokenUri())  
            .contentType(MediaType.APPLICATION_FORM_URLENCODED)
```

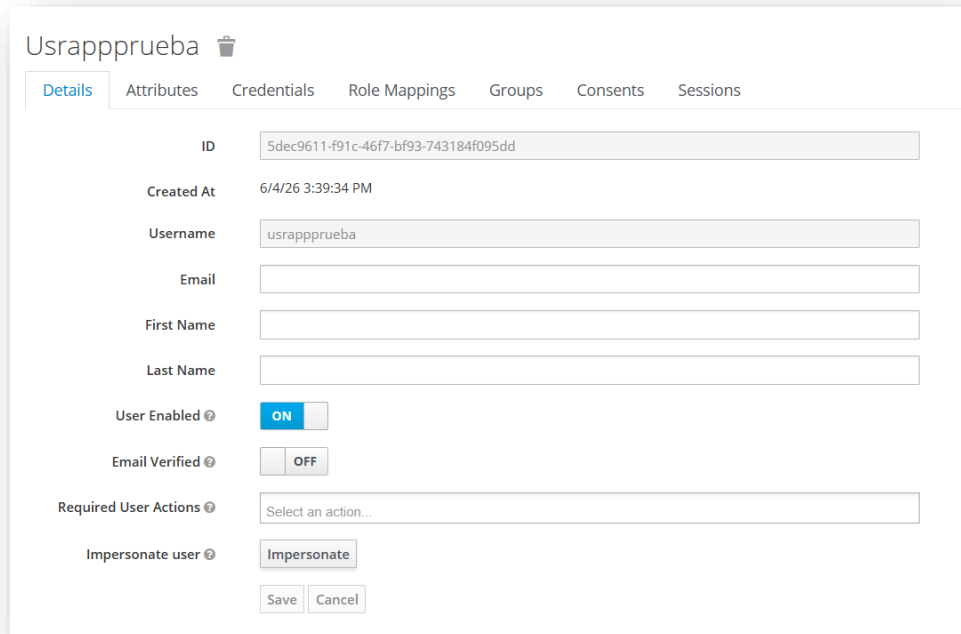


```
        .body(formData(username, password))
        .retrieve()
        .body(String.class);

        if (body == null || body.isBlank()) {
            throw new TokenException("Keycloak no devolvió
access_token");
        }

        JsonNode json = objectMapper.readTree(body);
        JsonNode accessToken = json.get("access_token");
        if (accessToken == null || accessToken.asText().isBlank()) {
            throw new TokenException("Respuesta de Keycloak sin
access_token");
        }
        return accessToken.asText();
    } catch (RestClientResponseException ex) {
        if (ex.getStatusCode().value() == 400 ||
ex.getStatusCode().value() == 401) {
            throw new AuthFailedException("Usuario o contraseña
incorrectos");
        }
        throw new TokenException("Error obteniendo token desde
Keycloak", ex);
    } catch (Exception ex) {
        throw new TokenException("Error obteniendo token desde
Keycloak", ex);
    }
}
```

Para este caso la aplicación consumidora tiene un usuario creado en el real OSIINTEGRACION que usa para generar un token propio.



The screenshot shows the 'Details' tab of a user profile in Keycloak. The user's ID is '5dec9611-f91c-46f7-bf93-743184f095dd', created at '6/4/26 3:39:34 PM', and the username is 'usrappprueba'. There are input fields for Email, First Name, and Last Name. The 'User Enabled' toggle is set to 'ON', and 'Email Verified' is 'OFF'. A dropdown for 'Required User Actions' shows 'Select an action...'. An 'Impersonate user' button is present, along with 'Save' and 'Cancel' buttons at the bottom.

Al pasarle el usuario y contraseña nos genera un token.

```
String token = keycloakTokenClient.obtenerAccessToken(username, password);
```

Luego al llamar a la aplicación que nos provee servicios tenemos que pasar el token el header como authorization.

```
public String obtenerClientes(String token) {  
    try {  
        String response = restClient.get()  
            .uri(properties.destino().obtenerClientesPath())  
            .header(HttpHeaders.AUTHORIZATION, "Bearer " + token)  
            .accept(MediaType.APPLICATION_JSON)  
            .retrieve()  
            .body(String.class);  
  
        if (response == null) {  
            throw new RemoteServiceException("La respuesta de app-  
destino fue vacia");  
        }  
        return response;  
    }  
}
```

```
    } catch (RestClientResponseException ex) {  
        throw new RemoteServiceException("Error HTTP " +  
ex.getStatusCode().value() + " al consumir app-destino", ex);  
    } catch (Exception ex) {  
        throw new RemoteServiceException("Timeout o error invocando app-  
destino", ex);  
    }  
}
```

12.5 ANEXO E: EJEMPLO DE APLICACIÓN QUE PROVEE SERVICIOS

El servicio que provee servicios tiene un endpoint de ejemplo llamado obtenerClientes, este tiene un filtro que validará los token que se le envíen por authorization.

```
@GetMapping("/api/obtenerClientes")  
public ResponseEntity<List<ClienteResponse>>  
obtenerClientes(HttpServletRequest request) {  
    try {  
        Object tokenAttribute =  
request.getAttribute(JwtAuthFilter.REQUEST_ATTRIBUTE_JWT_TOKEN);  
        Object rolesAttribute =  
request.getAttribute(JwtAuthFilter.REQUEST_ATTRIBUTE_JWT_ROLES);  
  
        if (tokenAttribute == null) {  
            throw new IllegalStateException("El filtro no dejó  
disponible el token validado");  
        }  
        if (!(rolesAttribute instanceof Set<?> roles) ||  
roles.isEmpty()) {  
            throw new IllegalStateException("El filtro no dejó  
disponibles los roles del JWT");  
        }  
  
        List<ClienteResponse> clientes =  
clienteService.obtenerClientes();  
        if (clientes == null) {  
            throw new IllegalStateException("La lista de clientes no  
puede ser nula");  
        }  
        return ResponseEntity.ok(clientes);  
    }  
}
```

```
    } catch (InvalidTokenException ex) {  
        throw ex;  
    } catch (Exception ex) {  
        throw new IllegalStateException("No fue posible obtener la lista  
de clientes", ex);  
    }  
}
```

Para este ejemplo se utilizo OncePerRequestFilter para validar los tokens que llegan. Después de validar con PTSD_VALIDAR_TOKEN, se extraen los roles y se busca que contenga el rol "rolivr".

```
@Component  
public class JwtAuthFilter extends OncePerRequestFilter {  
  
    public static final String REQUEST_ATTRIBUTE_JWT_TOKEN = "JWT_TOKEN";  
    public static final String REQUEST_ATTRIBUTE_JWT_ROLES = "JWT_ROLES";  
  
    private static final String REQUIRED_ROLE = "rolivr";  
    private static final String PROTECTED_PATH = "/api/obtenerClientes";  
  
    private final BearerTokenExtractor bearerTokenExtractor;  
    private final PtsdTokenValidatorClient ptsdTokenValidatorClient;  
    private final JwtUtils jwtUtils;  
    private final HandlerExceptionResolver handlerExceptionResolver;  
  
    public JwtAuthFilter(BearerTokenExtractor bearerTokenExtractor,  
                        PtsdTokenValidatorClient ptsdTokenValidatorClient,  
                        JwtUtils jwtUtils,  
                        @Qualifier("handlerExceptionResolver")  
HandlerExceptionResolver handlerExceptionResolver) {  
        this.bearerTokenExtractor = bearerTokenExtractor;  
        this.ptsdTokenValidatorClient = ptsdTokenValidatorClient;  
        this.jwtUtils = jwtUtils;  
        this.handlerExceptionResolver = handlerExceptionResolver;  
    }  
  
    @Override  
    protected boolean shouldNotFilter(HttpServletRequest request) {  
        return !request.getRequestURI().endsWith(PROTECTED_PATH);  
    }  
}
```

```
@Override
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain filterChain)
    throws ServletException, IOException {
    try {
        String token =
bearerTokenExtractor.extract(request.getHeader("Authorization"));
        ptsdTokenValidatorClient.validateToken(token);
        Set<String> roles = jwtUtils.extractRoles(token);

        if (!roles.contains(REQUIRED_ROLE)) {
            throw new MissingRolesException("El usuario no posee el rol
requerido: " + REQUIRED_ROLE);
        }

        request.setAttribute(REQUEST_ATTRIBUTE_JWT_TOKEN, token);
        request.setAttribute(REQUEST_ATTRIBUTE_JWT_ROLES, roles);
        filterChain.doFilter(request, response);
    } catch (AuthorizationHeaderMissingException |
BearerTokenFormatException | UnauthorizedTokenException |
MissingRolesException | InvalidTokenException |
ExternalServiceException ex) {
        handlerExceptionResolver.resolveException(request, response,
null, ex);
    } catch (Exception ex) {
        handlerExceptionResolver.resolveException(request, response,
null, ex);
    }
}
```

En mitad de la validación se llama a PTSD_VALIDAR_TOKEN.

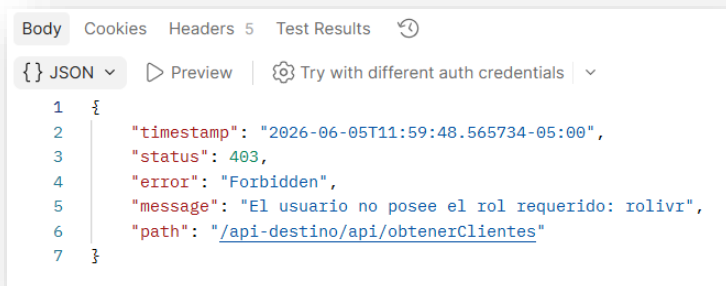
```
@Override
public void validateToken(String token) {
    try {
        webClient.get()
            .uri(appProperties.ptsd().validateUrl())
            .header(HttpHeaders.AUTHORIZATION, "Bearer " + token)
            .retrieve()
```

```
        .onStatus(HttpStatusCode::is4xxClientError, response ->
response.statusCode().value() == 401
            ? response.bodyToMono(String.class).map(body ->
new UnauthorizedTokenException("El token no es válido"))
            : response.bodyToMono(String.class).map(body ->
new ExternalServiceException("PTSD_VALIDAR_TOKEN rechazó la solicitud",
null)))

        .onStatus(HttpStatusCode::is5xxServerError, response ->
response.bodyToMono(String.class).map(body -> new
ExternalServiceException("Error del servicio PTSD_VALIDAR_TOKEN", null)))
        .toBodilessEntity()
        .block();
    } catch (WebClientResponseException e) {
        if (e.getStatusCode().value() == 401) {
            throw new UnauthorizedTokenException("El token no es
válido");
        }
        throw new ExternalServiceException("Error comunicándose con
PTSD_VALIDAR_TOKEN", e);
    } catch (ExternalServiceException e) {
        throw e;
    } catch (Exception e) {
        throw new ExternalServiceException("Error comunicándose con
PTSD_VALIDAR_TOKEN", e);
    }
}
```

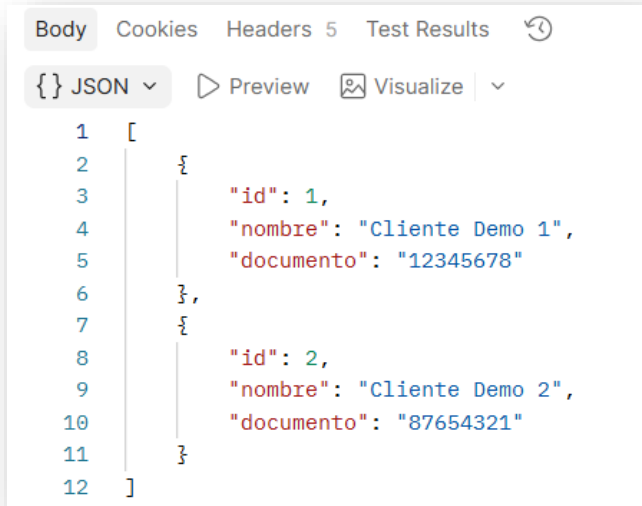
Ejemplos de respuesta del servicio.

Cuando el usuario no tiene el rol correspondiente.



```
{
  "timestamp": "2026-06-05T11:59:48.565734-05:00",
  "status": 403,
  "error": "Forbidden",
  "message": "El usuario no posee el rol requerido: rolivr",
  "path": "/api-destino/api/obtenerClientes"
}
```

Cuando el usuario si tiene el rol “rolivr”



```
1  [
2    {
3      "id": 1,
4      "nombre": "Cliente Demo 1",
5      "documento": "12345678"
6    },
7    {
8      "id": 2,
9      "nombre": "Cliente Demo 2",
10     "documento": "87654321"
11   }
12 ]
```